

Tech Price Pipeline

Monitoreo de Precios de Laptops Refurbished

Rolando Suárez

Data Engineer

Resumen

Este documento describe el pipeline de Ingeniería de Datos desarrollado para el monitoreo automático de precios de laptops refurbished en las plataformas Amazon, eBay y Newegg. El sistema implementa una arquitectura ETL completa, desde la extracción de datos mediante técnicas avanzadas de web scraping, pasando por la transformación y normalización de la información, hasta la visualización interactiva en un dashboard desarrollado con Streamlit. El proyecto incluye un orquestador principal (`run_pipeline.py`) que unifica todos los componentes en una interfaz de menú interactivo.

1. Resumen del Proyecto

El pipeline extrae información en tiempo real de tres plataformas principales de comercio electrónico, procesa los datos bajo estándares de calidad y los presenta en una interfaz gráfica interactiva. El sistema está diseñado específicamente para el monitoreo de laptops refurbished (aunque los scrapers pueden extraer cualquier producto de estas paginas), extrayendo características clave como marca, memoria RAM y capacidad de almacenamiento.

1.1. Características Principales

- **Extracción automatizada** con `undetected-chromedriver` (evita bloqueos anti-bot)
- **Filtrado geográfico** (eBay: solo productos de USA con precios en USD)
- **Transformación ETL** con limpieza, normalización y deduplicación de datos
- **Arquitectura por capas**: Raw (JSON) → Silver (CSV) → Gold (SQLite)
- **Dashboard interactivo** con Streamlit para análisis comparativo
- **Orquestador central** con menú interactivo para ejecutar cualquier combinación de componentes

2. Arquitectura del Pipeline

El sistema se organiza en cuatro capas principales:

2.1. Capa 1: Extracción (Scrapers)

Tres scrapers independientes utilizan `undetected-chromedriver` para evitar la detección:

- **Amazon**: Configura ubicación en Miami (código postal 33101)
- **eBay**: Filtra productos de USA y precios en USD mediante parámetros URL
- **Newegg**: Extracción estándar con selectores específicos

2.2. Capa 2: Almacenamiento Raw (JSON)

Cada ejecución genera un archivo JSON con timestamp único en `data/raw/*.json`

2.3. Capa 3: Transformación (ETL)

El script `data_cleaning.py` ejecuta:

1. Limpieza de precios (elimina monedas y caracteres no numéricos)
2. Extracción de RAM mediante patrones regex
3. Extracción de almacenamiento (convierte TB a GB automáticamente)
4. Detección de marca (APPLE, DELL, HP, LENOVO, ASUS, ACER, SAMSUNG, MICROSOFT, MSI, GIGABYTE)
5. Deduplicación por URL y por título+plataforma

2.4. Capa 4: Almacenamiento Final

- Silver (CSV): Respaldo de datos limpios en `data/silver/`
- Gold (SQLite): Base de datos final en `data/db/tech_prices_gold.db`

3. Estructura del Proyecto

```
1 tech-price-pipeline/
2
3     .gitignore                # Archivos excluidos del repositorio
4     README.md                 # Documentacion principal
5     requirements.txt          # Dependencias del proyecto
6     run_pipeline.py           # ORQUESTADOR PRINCIPAL (menu
interactivo)
7
8     scrapers/                 # Modulo de extraccion
9         amazon_monitor.py     # Scraper Amazon (con cambio de
ubicacion a Miami)
10        ebay_monitor.py       # Scraper eBay (filtro USA + USD)
11        newegg_monitor.py     # Scraper Newegg
12
13    processing/                # Modulo de transformacion
14        data_cleaning.py       # ETL completo: limpieza,
normalizacion y carga
15
16    visualization/             # Modulo de visualizacion
17        dashboard.py           # Dashboard interactivo Streamlit
18
19    data/                       # Datos (generados localmente, NO se
sube a GitHub)
20        raw_amazon_*.json      # Capa Raw: datos crudos de Amazon
21        raw_ebay_*.json        # Capa Raw: datos crudos de eBay
22        raw_newegg_*.json      # Capa Raw: datos crudos de Newegg
23        silver/                # Capa Silver: respaldo en CSV
24            silver_tech_prices_*.csv
25            silver_tech_prices_latest.csv
26        db/                    # Capa Gold: base de datos final
27        tech_prices_gold.db     # SQLite con productos unificados
```

Listing 1: Estructura completa de directorios

4. Orquestador Principal (run_pipeline.py)

El archivo `run_pipeline.py` es el punto de entrada unificado del sistema. Implementa una clase `PipelineOrchestrator` que organiza y ejecuta todos los componentes del pipeline.

4.1. Funcionalidades del Orquestador

- **Menu interactivo:** Permite seleccionar qué componentes ejecutar
- **Ejecución secuencial:** Garantiza el orden correcto de ejecución
- **Manejo de errores:** Captura y reporta fallos en cada componente
- **Validación de dependencias:** Verifica que existan los archivos necesarios
- **Resumen final:** Muestra el estado de cada componente ejecutado

4.2. Metodos del Orquestador

Metodo	Descripción
<code>run_amazon()</code>	Ejecuta el scraper de Amazon
<code>run_ebay()</code>	Ejecuta el scraper de eBay
<code>run_newegg()</code>	Ejecuta el scraper de Newegg
<code>run_etl()</code>	Ejecuta el procesamiento ETL (<code>data_cleaning.py</code>)
<code>run_dashboard()</code>	Lanza el dashboard de Streamlit
<code>run_pipeline_completo()</code>	Ejecuta todos los componentes en secuencia

Cuadro 1: Metodos principales del orquestador

4.3. Menu de Opciones

```
1 === Tech Price Pipeline - Orquestador ===
2 1. Ejecutar pipeline completo (Amazon + eBay + Newegg + ETL + Dashboard)
3 2. Ejecutar solo Amazon + ETL + Dashboard
4 3. Ejecutar solo eBay + ETL + Dashboard
5 4. Ejecutar solo Newegg + ETL + Dashboard
6 5. Ejecutar solo ETL (procesar JSON existentes)
7 6. Ejecutar solo Dashboard
8 7. Ejecutar solo Amazon
9 8. Ejecutar solo eBay
10 9. Ejecutar solo Newegg
11 10. Salir
```

Listing 2: Menu interactivo del orquestador

5. Tecnologías Utilizadas

Categoría	Tecnologías
Lenguaje	Python 3.12+
Web Scraping	Selenium, Undetected-Chromedriver
Data Processing	Pandas, Numpy, Regex
Base de Datos	SQLite3
Visualización	Streamlit
Sistema Operativo	Linux Mint (compatible con Mac/Windows)

Cuadro 2: Tecnologías empleadas en el pipeline

6. Instalación Paso a Paso

6.1. Requisitos Previos

- Python 3.12 o superior
- Google Chrome o Firefox instalado
- Git (opcional, para clonar)

6.2. Pasos de Instalación

```
1 # Clonar el repositorio
2 git clone https://github.com/RolandoSu03/tech-price-pipeline.git
3 cd tech-price-pipeline
4
5 # Crear y activar entorno virtual (Linux/Mac)
6 python3 -m venv venv
7 source venv/bin/activate
8
9 # Instalar dependencias
10 pip install -r requirements.txt
```

Listing 3: Clonar y configurar entorno

6.3. Verificar Instalación del Navegador

```
1 # Para Chrome
2 google-chrome --version
3
4 # Para Firefox
5 firefox --version
```

Listing 4: Verificar navegador

7. Uso del Pipeline

7.1. Ejecutar el Orquestador (Recomendado)

```

1 # Activar entorno virtual
2 source venv/bin/activate
3
4 # Ejecutar orquestador
5 python run_pipeline.py
6
7 # Seleccionar una opcion del menu

```

Listing 5: Ejecutar pipeline con menu interactivo

7.2. Ejecutar Componentes Individuales

```

1 # Ejecutar solo un scraper
2 python scrapers/amazon_monitor.py
3 python scrapers/ebay_monitor.py
4 python scrapers/newegg_monitor.py
5
6 # Ejecutar solo el procesamiento ETL
7 python processing/data_cleaning.py
8
9 # Ejecutar solo el dashboard
10 streamlit run visualization/dashboard.py

```

Listing 6: Ejecucion directa de componentes

8. Formato de Datos

8.1. Capa Raw (JSON)

```

1 {
2     "title": "Dell Latitude 7490 16GB RAM 512GB SSD",
3     "price_raw": "$329.99",
4     "url": "https://www.ebay.com/itm/123456789",
5     "platform": "ebay",
6     "scraped_at": "2025-05-10 15:30:22"
7 }

```

Listing 7: Ejemplo de JSON crudo

8.2. Capa Silver (CSV) - Respaldo

La carpeta data/silver/ contiene archivos CSV con datos ya limpios y normalizados. Esta capa sirve como:

- **Respaldo intermedio:** Si la base de datos falla, los datos estructurados están disponibles
- **Verificación rápida:** Se puede abrir con Excel o cualquier editor de texto
- **Compatibilidad:** Los CSV pueden usarse en otras herramientas sin necesidad de SQL

8.3. Capa Gold (SQLite)

Columna	Tipo	Descripción
id	INTEGER	Clave primaria autoincremental
title	TEXT	Título original del producto
brand	TEXT	Marca detectada (o "OTHER")
ram_gb	INTEGER	RAM en GB (ej: 16, 32, 64)
storage_gb	INTEGER	Almacenamiento en GB (ej: 256, 512, 1024)
price	REAL	Precio numérico limpio
url	TEXT	URL única del producto
platform	TEXT	Plataforma (amazon, ebay, newegg)
scraped_at	DATETIME	Fecha y hora de extracción
last_updated	DATETIME	Fecha de última actualización

Cuadro 3: Estructura de la tabla `products` en SQLite

9. Filtrado Geográfico por Plataforma

Plataforma	Filtro USA	Método
eBay	SI	Parámetros URL (<code>LH_PrefLoc=1, _salic=1</code>) + validación de moneda
Amazon	PARCIAL	Cambio de ubicación a Miami (código postal 33101)
Newegg	NO	Sin filtros específicos

Cuadro 4: Filtrado geográfico por plataforma

10. Solución de Problemas Comunes

10.1. Error 403 en eBay (bloqueo)

```
1 # En ebay_monitor.py, aumentar los delays
2 time.sleep(random.uniform(8, 12)) # Linea aproximada 45
```

Listing 8: Aumentar tiempos de espera

10.2. No se detectan RAM o Storage

Los patrones regex buscan formatos estándar como "16GB RAM." o "512GB SSD". Si los productos usan formatos diferentes, modificar los patrones en `data_cleaning.py`:

```
1 patterns = [
2     r'(\d+)\s*GB\s*RAM',          # "16GB RAM"
3     r'RAM\s*(\d+)\s*GB',          # "RAM 16GB"
4     r'(\d+)\s*GB\s*SSD',          # "256GB SSD"
5 ]
```

Listing 9: Patrones de extracción

10.3. Chrome/Firefox no encontrado

```
1 wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | sudo apt-key add -
2 sudo sh -c 'echo "deb [arch=amd64] http://dl.google.com/linux/chrome/deb/ stable
   main" >> /etc/apt/sources.list.d/google-chrome.list'
```

```
3 sudo apt update && sudo apt install google-chrome-stable
```

Listing 10: Instalar Chrome en Linux

10.4. El dashboard no muestra datos

```
1 # Verificar que la base de datos existe
2 ls -la data/db/tech_prices_gold.db
3
4 # Verificar que tiene datos
5 sqlite3 data/db/tech_prices_gold.db "SELECT COUNT(*) FROM products;"
```

Listing 11: Verificar base de datos

11. Mejoras Sugeridas

- **Orquestación con Airflow:** Programar ejecuciones diarias automáticas
- **Alertas por email:** Notificar cuando un producto baja de precio
- **Despliegue en la nube:** Render o Streamlit Cloud para el dashboard
- **Logging estructurado:** Reemplazar `print()` por logs con niveles (INFO, ERROR)
- **Soporte para más plataformas:** Best Buy, B&H, Walmart
- **Histórico de precios:** Guardar cambios de precio en el tiempo
- **Filtro USA para Amazon y Newegg:** Agregar parámetros adicionales en las URLs

Créditos

Rolando Suárez

Data Engineer

- GitHub: <https://github.com/RolandoSu03>
- Repositorio del proyecto: <https://github.com/RolandoSu03/tech-price-pipeline>

Nota: Este proyecto respeta los términos de servicio de las plataformas web y es exclusivamente para fines educativos y de portafolio.